DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING

COLLEGE OF E&ME, NUST, RAWALPINDI

EC312 Digital Image Processing
Assignment 1

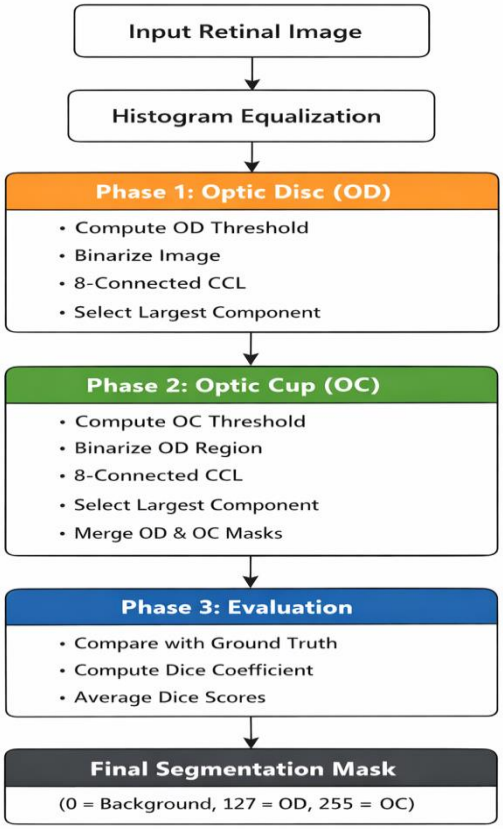
SUBMITTED TO:
Dr. Asad Mansoor

SUBMITTED BY:
Fatima Masood
Reg # 456153
DE- 45-B Dept CE

Submission Date: 16 February 2026

Retinal fundus Image Segmentation using Connected Component Labeling

Flow diagram:



Approach:

Phase 1: Optic Disc (OD) Segmentation

- Convert the image to grayscale and apply histogram equalization to enhance contrast.
- From training images, collect OD pixel intensities from ground truth masks.
- Compute a threshold (28th percentile) to select OD candidate pixels (V-set).
- Binarize the image using this threshold and apply 8-connected component labeling.
- Select the largest connected component as the final OD mask.

Phase 2: Optic Cup (OC) Segmentation

- Inside the OD region, collect pixel intensities of the cup from training masks.
- Compute a higher threshold (65th percentile) to select OC pixels (V-set).
- Binarize OD region using this threshold and apply 8-connected CCL.
- Select the largest connected component as the OC mask.
- Combine OD and OC into a single mask: background = 0, OD = 127, OC = 255.

Phase 3: Compute Dice Coefficient

- Compare predicted masks with ground truth for background, OD, and OC.
- Compute Dice coefficient for each class:

$$\text{Dice} = \frac{2 \times \text{overlap pixels}}{\text{predicted pixels} + \text{ground truth pixels}}$$

- Average Dice values over all test images to measure performance.

Codes

CC8 Editable Code:

```
import numpy as np
import cv2

def eight_connected_components(image):
    rows, cols = image.shape
    output = np.zeros((rows, cols), dtype=np.uint16)
    count = 0
    equivalence = {}

    # First Pass
    for i in range(rows):
        for j in range(cols):

            if image[i, j] !=0:
```

```
if i >= 1:
    top = output[i-1, j]
else:
    top = 0

if j >= 1:
    left = output[i, j-1]
else:
    left = 0

if i > 0 and j > 0:
    top_left = output[i-1, j-1]
else:
    top_left = 0

if i > 0 and j < cols-1:
    top_right = output[i-1, j+1]
else:
    top_right = 0

# Case 1: No neighbors
if top == 0 and left == 0 and top_left == 0 and top_right == 0:
    count += 1
    output[i, j] = count
    equivalence[count] = count

else:
    # Finding minimum non-zero neighbor
    min_label = 0

    neighbors = [left, top, top_left, top_right]

    for n in neighbors:
        if n != 0:
            if min_label == 0:
                min_label = n
            elif n < min_label:
                min_label = n

    output[i, j] = min_label
```

```
                # Storing in equivalency list
                for n in neighbors:
                    if n != 0 and n != min_label:
                        equivalence[n] = min_label

    # Second Pass
    for i in range(rows):
        for j in range(cols):
            if output[i, j] != 0:
                label = output[i, j]

                while equivalence[label] != label:
                    label = equivalence[label]

                output[i, j] = label

    # Count objects
    objects = output[output > 0]
    unique_objects = np.unique(objects)
    Total_objects = len(unique_objects)

    return output


if __name__ == '__main__':
    image = cv2.imread("x_image.png", 0)
    labeled_image = eight_connected_components(image)
```

Phase 1 (Optic_Disk V_Set)

Editable Code:

```
import numpy as np
import cv2
import os
from main import eight_connected_components
```

```
# Taking threshold from training images
def Compute_OD_threshold(folder_path, mask_folder):
    all_pixels = []
    images = os.listdir(folder_path)

    for im in images:
        img = cv2.imread(os.path.join(folder_path, im), 0)
        img = cv2.equalizeHist(img)

        name = os.path.splitext(im)[0]
        mask_path = os.path.join(mask_folder, name, "SoftMap", name + "_ODsegSoftmap.png")
        mask = cv2.imread(mask_path, 0)

        if img is None or mask is None:
            continue

        OD_pixels = img[mask > 0]
        all_pixels.extend(OD_pixels)

    all_pixels = np.array(all_pixels)
    # Using percentile to avoid noise
    T = np.percentile(all_pixels, 28)
    print("OD Threshold:", T)
    return int(T)


# Segment OD from image
def segment_OD(image, T):
    binary = np.zeros(image.shape,dtype=np.uint8)
    binary[image >= T] = 255

    labels = eight_connected_components(binary)
    # find largest component
    unique, counts = np.unique(labels[labels > 0], return_counts=True)
    largest = unique[np.argmax(counts)]

    OD_mask = np.zeros(image.shape,dtype=np.uint8)
    OD_mask[labels == largest] = 255

    return OD_mask
```

```
if __name__ == "__main__":
    train_img = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Training\\Training\\Images\\NORMAL"
    train_gt = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Training\\Training\\GT"
    test_folder = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Test\\Test\\Images\\normal"

# OD V-set threshold
    T = Compute_OD_threshold(train_img, train_gt)

# Run on test images
    for im in os.listdir(test_folder):
        img = cv2.imread(os.path.join(test_folder, im), 0)
        img = cv2.equalizeHist(img)

        OD = segment_OD(img, T)

        cv2.imshow("Original", cv2.resize(img,(512,512)))
        cv2.imshow("OD Segmented", cv2.resize(OD,(512,512)))
        cv2.waitKey(0)
```

Phase 2 (Optic_Cup V_Set & 3_label_image)

Editable Code:

```
import numpy as np
import cv2
import os
from OD_Part import Compute_OD_threshold, segment_OD
from main import eight_connected_components

def Compute_OC_threshold(folder_path, mask_folder):
    all_pixels = []
    images = os.listdir(folder_path)

    for im in images:
```

```
        img = cv2.imread(os.path.join(folder_path, im), 0)
        img = cv2.equalizeHist(img)

        name = os.path.splitext(im)[0]
        cup_mask_path = os.path.join(mask_folder, name, "SoftMap", name +
"_cupsegSoftmap.png")
        mask = cv2.imread(cup_mask_path, 0)

        if img is None or mask is None:
            continue

        OC_pixels = img[mask > 0]
        all_pixels.extend(OC_pixels)

    all_pixels = np.array(all_pixels)
    T = np.percentile(all_pixels, 65)
    print("OC Threshold:", T)
    return int(T)

def segment_OC(image, OD_mask, T_OC):
    binary = np.zeros(image.shape,dtype=np.uint8)
    binary[(image >= T_OC) & (OD_mask > 0)] = 255

    labels = eight_connected_components(binary)
    objs = labels[labels > 0]

    if len(objs)==0:
        return np.zeros(image.shape,dtype=np.uint8)

    unique, counts = np.unique(objs, return_counts=True)
    largest = unique[np.argmax(counts)]

    OC_mask = np.zeros(image.shape,dtype=np.uint8)
    OC_mask[labels == largest] = 255

    return OC_mask

if __name__ == "__main__":
    train_img = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-
GS\\Training\\Training\\Images\\NORMAL"
```

```
    train_gt = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Training\\Training\\GT"
    test_folder = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Test\\Test\\Images\\normal"

    T_OD = Compute_OD_threshold(train_img, train_gt)
    T_OC = Compute_OC_threshold(train_img, train_gt)

# Run on test images
    test_images = os.listdir(test_folder)
    for im in test_images:
        img = cv2.imread(os.path.join(test_folder, im), 0)
        img = cv2.equalizeHist(img)

        OD = segment_OD(img, T_OD)
        cv2.imshow("OD image", cv2.resize(OD, (512, 512)))

        OC = segment_OC(img, OD, T_OC)
        cv2.imshow("OC image", cv2.resize(OC, (512, 512)))

        final = np.zeros(img.shape,dtype=np.uint8)
        final[OD > 0] = 127
        final[OC > 0] = 255

        cv2.imshow("Final", cv2.resize(final, (512, 512)))
        cv2.waitKey(0)
```

Phase 3 (Dice Coefficient):

Editable Code:

```
import numpy as np
import cv2
import os
from OD_Part import Compute_OD_threshold, segment_OD
from Phase2 import Compute_OC_threshold, segment_OC

def Disc_Coeff(pred, gt):
    pred = pred > 0
```

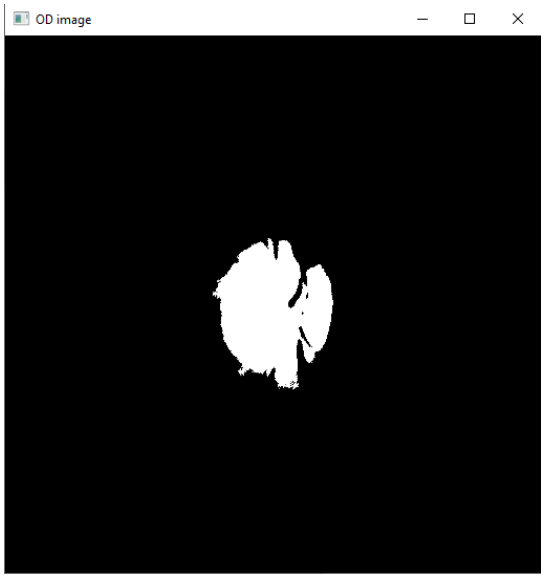
```
    gt = gt > 0
    intersection = np.sum(pred & gt)
    total = np.sum(pred) + np.sum(gt)
    if total == 0:
        return 1.0  # so no infinite case
    disc_coef = 2 * intersection / total
    return disc_coef

if __name__ == "__main__":
    # Paths
    train_img = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Training\\Training\\Images\\NORMAL"
    train_gt = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Training\\Training\\GT"
    test_folder = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Test\\Test\\Images\\normal"
    test_gt = "G:\\Computer Engineering\\6th Semester\\DIP\\Assignment\\Drishti-GS\\Drishti-GS\\Test\\Test\\Test_GT"

    # Compute thresholds from training images
    T_OD = Compute_OD_threshold(train_img, train_gt)
    T_OC = Compute_OC_threshold(train_img, train_gt)

    # Initialize for average Dice
    total_bg = 0
    total_od = 0
    total_oc = 0
    count = 0
# Run on test images
    test_images = os.listdir(test_folder)
    for im in test_images:
        img_path = os.path.join(test_folder, im)
        img = cv2.imread(img_path, 0)
        if img is None:
            continue
        img = cv2.equalizeHist(img)

        # Segment OD and OC
        OD_image = segment_OD(img, T_OD)
        cv2.imshow("OD image", cv2.resize(OD_image, (512, 512)))
```

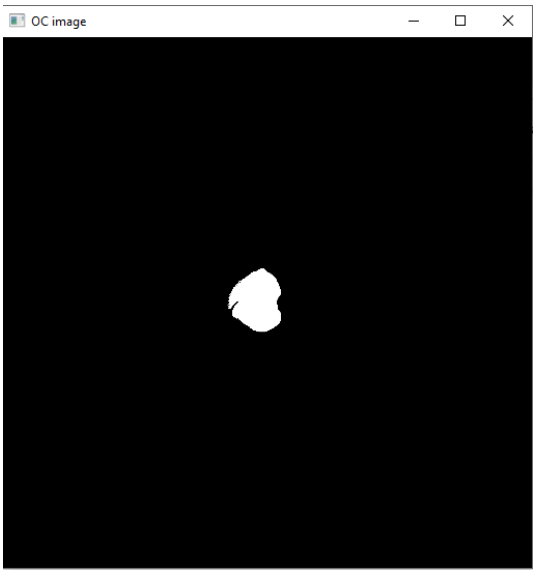
```
    OC_image = segment_OC(img, OD_image, T_OC)
    cv2.imshow("OC image", cv2.resize(OC_image, (512, 512)))

    # Create final mask
    final = np.zeros(img.shape, dtype=np.uint8)
    final[OD_image > 0] = 127
    final[OC_image > 0] = 255
    cv2.imshow("Final", cv2.resize(final, (512, 512)))
    cv2.waitKey(0)

    # Load ground-truth masks
    name = os.path.splitext(im)[0]
    gt_od_path = os.path.join(test_gt, name, "SoftMap", name + "_ODsegSoftmap.png")
    gt_oc_path = os.path.join(test_gt, name, "SoftMap", name + "_cupsegSoftmap.png")
    gt_od = cv2.imread(gt_od_path, 0)
    gt_oc = cv2.imread(gt_oc_path, 0)
    if gt_od is None or gt_oc is None:
        print(f"No ground truth  for {im}")
        continue

    # Compute Dice
    d_od = Disc_Coeff(OD_image, gt_od)
    d_oc = Disc_Coeff(OC_image, gt_oc)
    bg_pred = final == 0
    bg_gt = (gt_od == 0) & (gt_oc == 0)
    d_bg = Disc_Coeff(bg_pred, bg_gt)

    # Summation for average
    total_bg += d_bg
    total_od += d_od
    total_oc += d_oc
    count += 1

    # print each image Dice coefficient
    print(f"{im} → BG Dice: {d_bg:.2f},   OD Dice: {d_od:.2f},   OC Dice: {d_oc:.2f}")

# print average Dice

avg_bg = total_bg / count
```

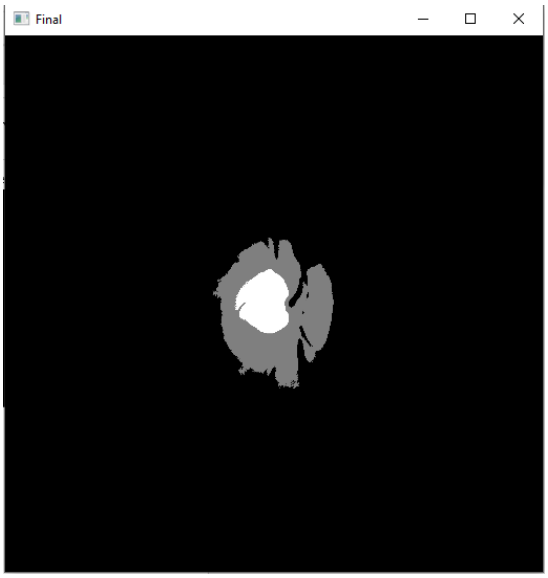
```
avg_od = total_od / count
avg_oc = total_oc / count
print("\nAverage Dice of Test Images:")
print(f"BG: {avg_bg*100:.2f}%,   OD: {avg_od*100:.2f}%,   OC: {avg_oc*100:.2f}%")
```

Output of Test Normal Images :

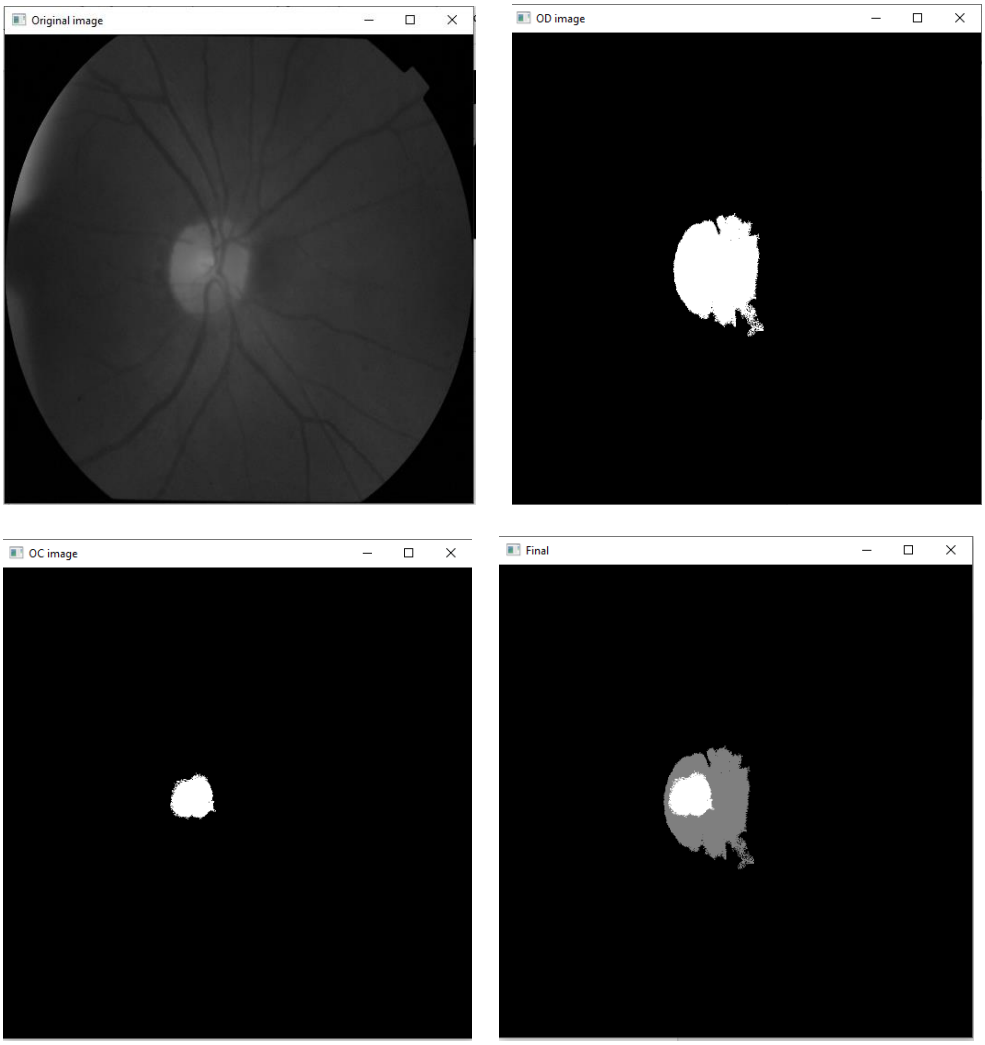
drishtiGS_007



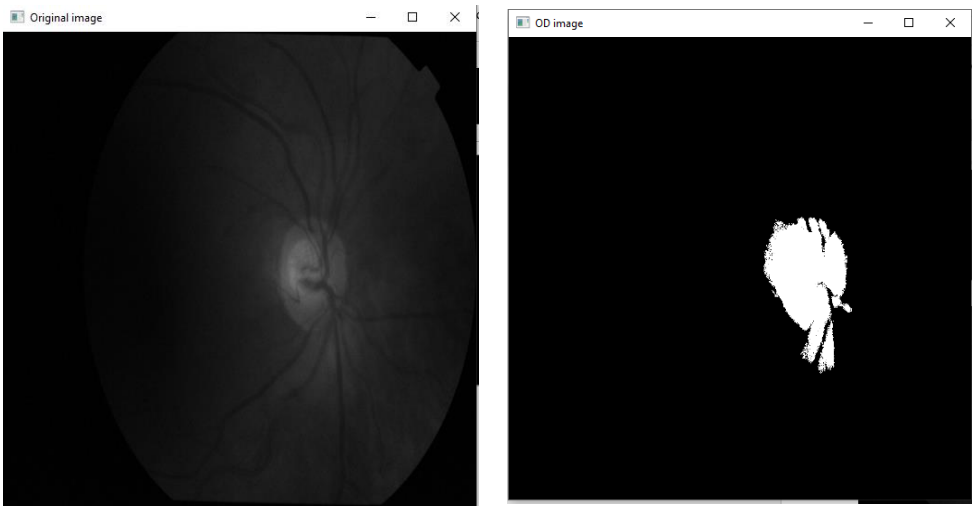


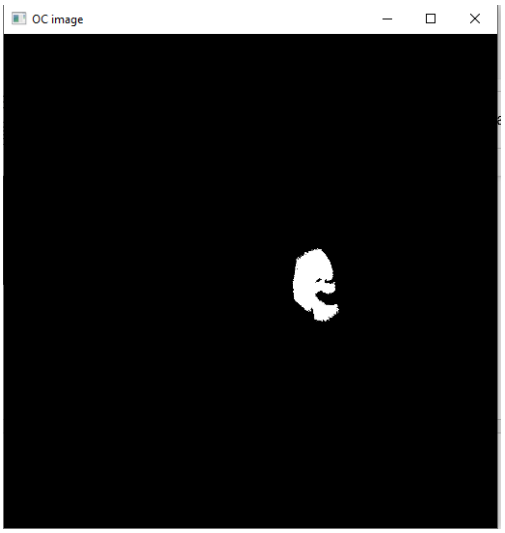


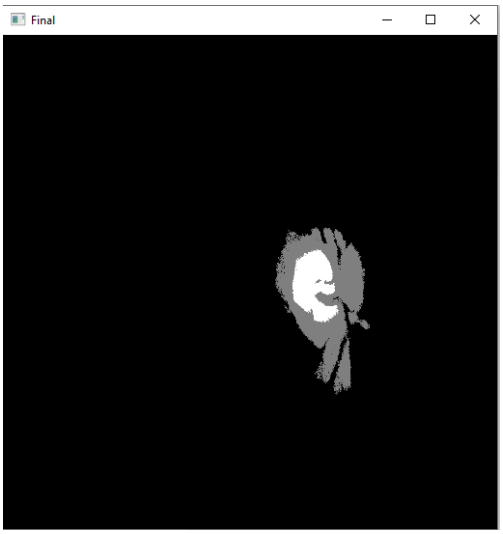
drishtiGS_009



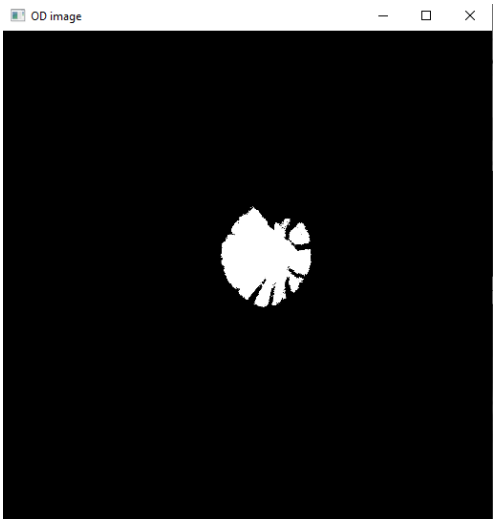
drishtiGS_013

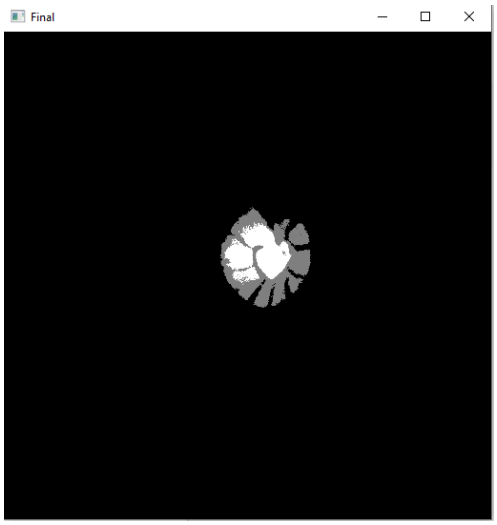




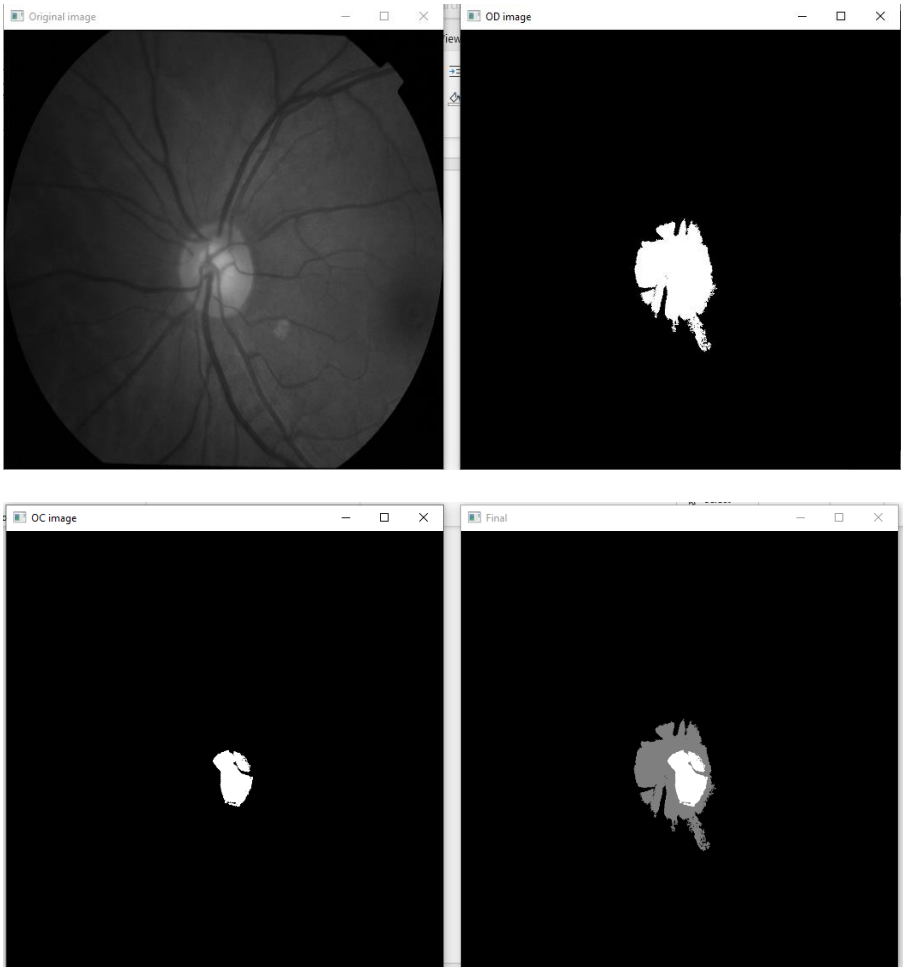


drishtiGS_072

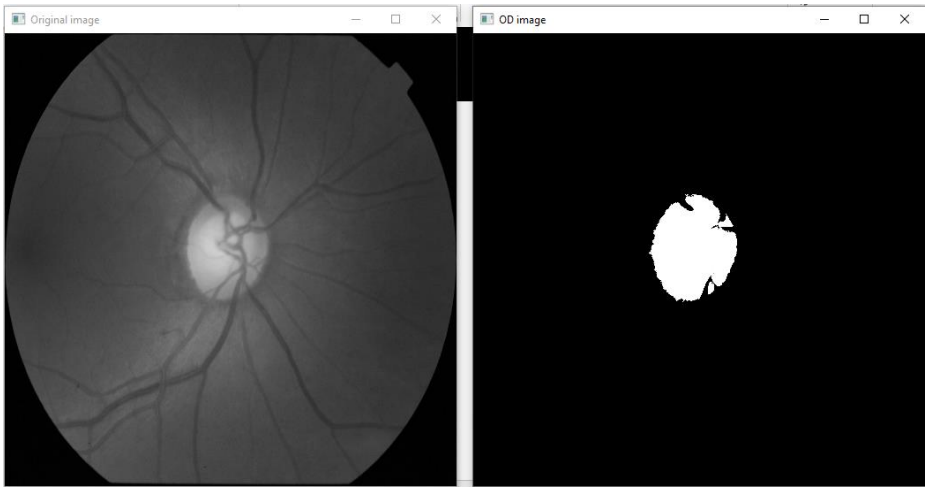


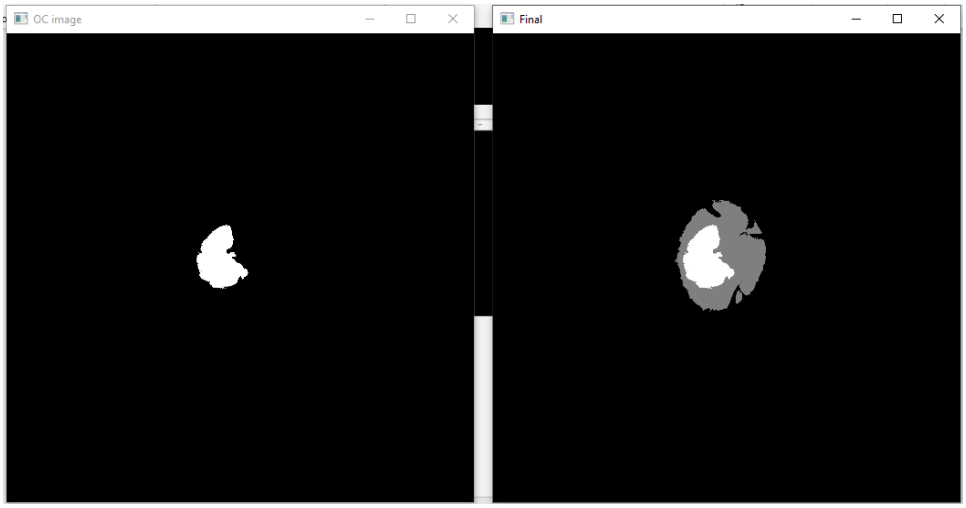


drishtiGS_078

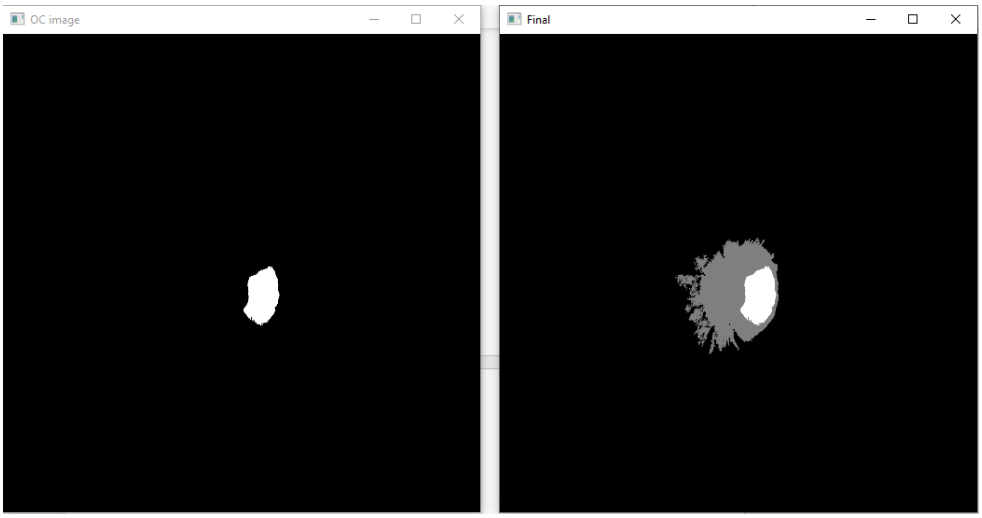


drishtiGS_085

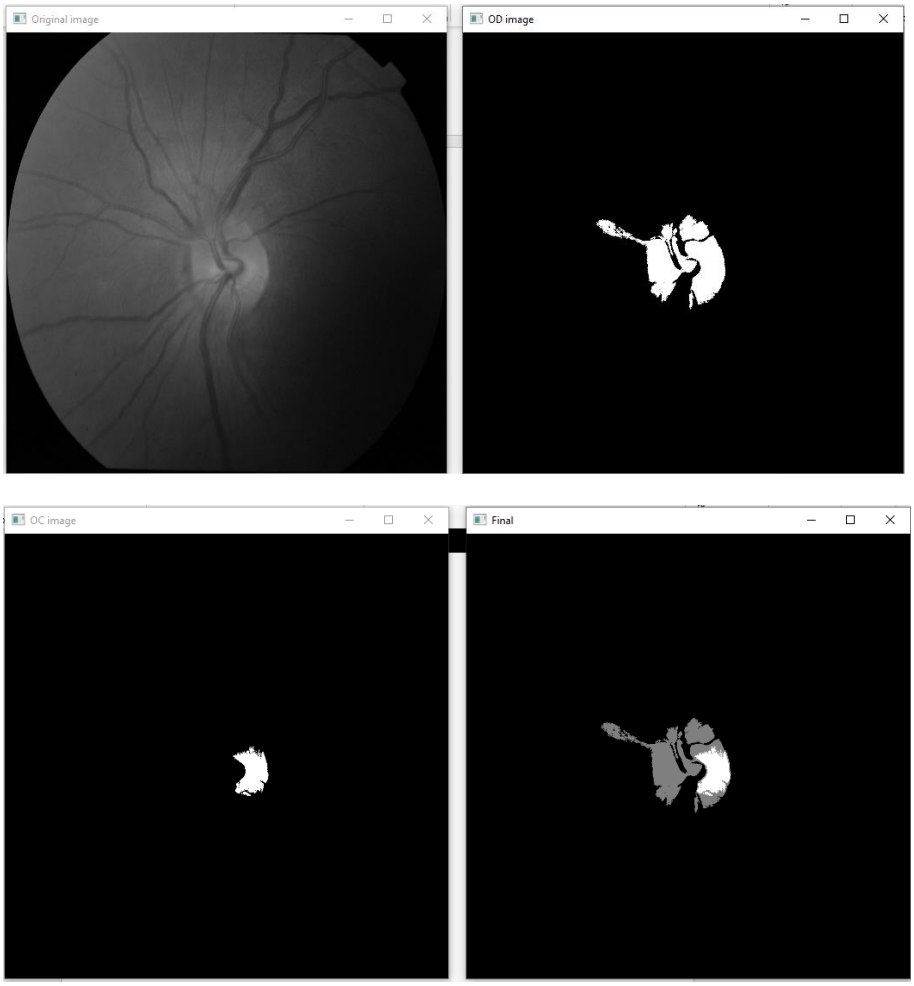




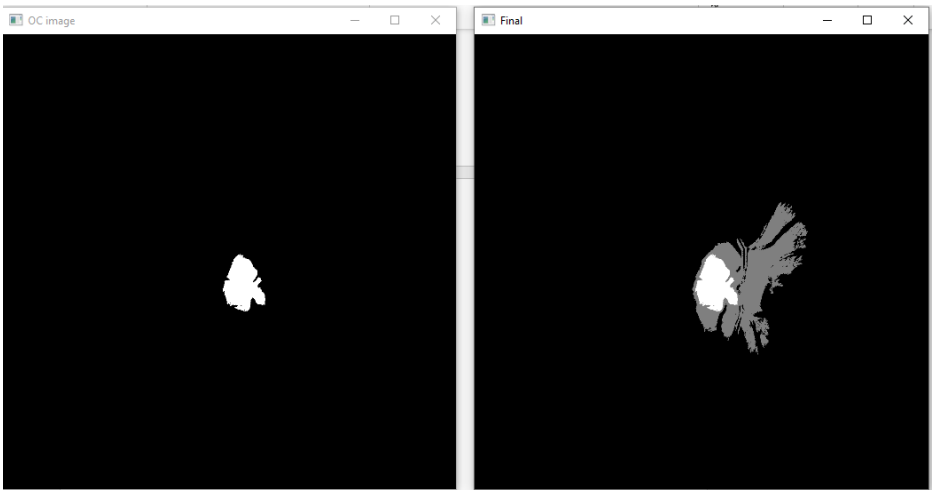
drishtiGS_091



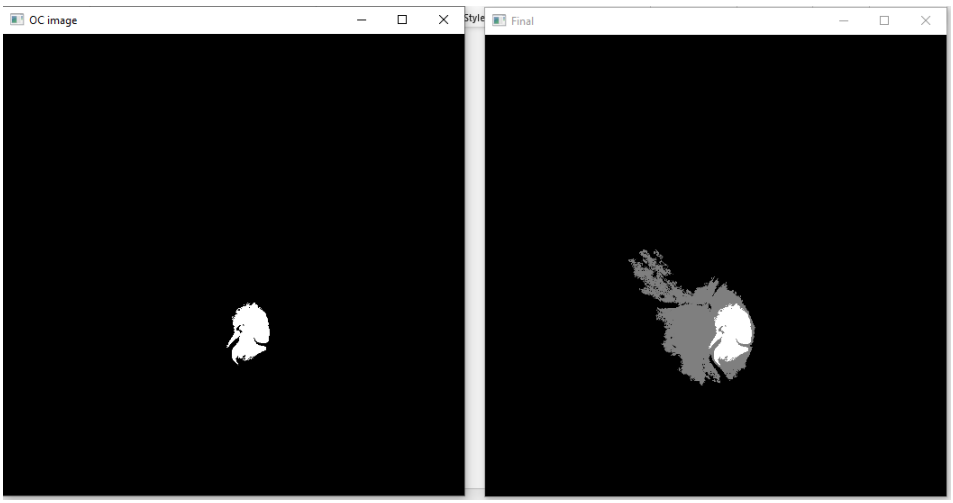
drishtiGS_093



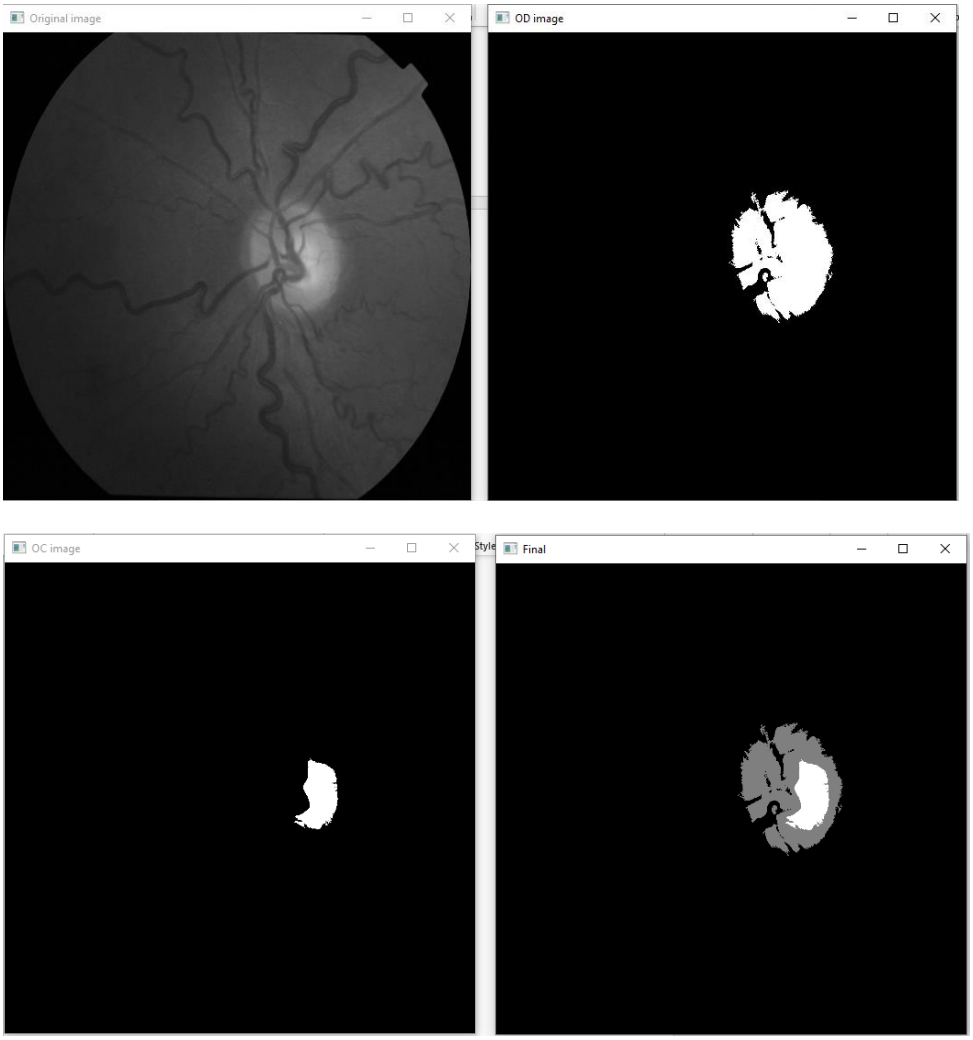
drishtiGS_095



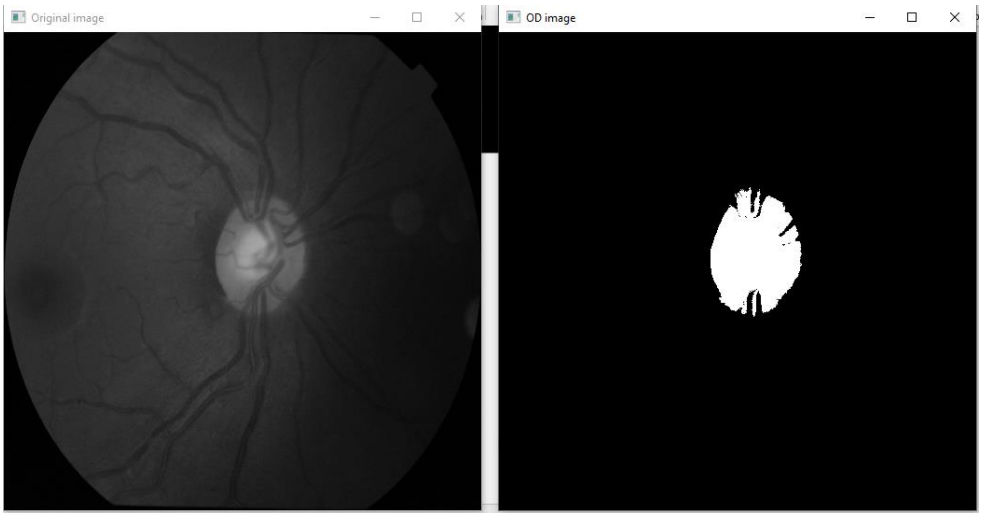
drishtiGS_096

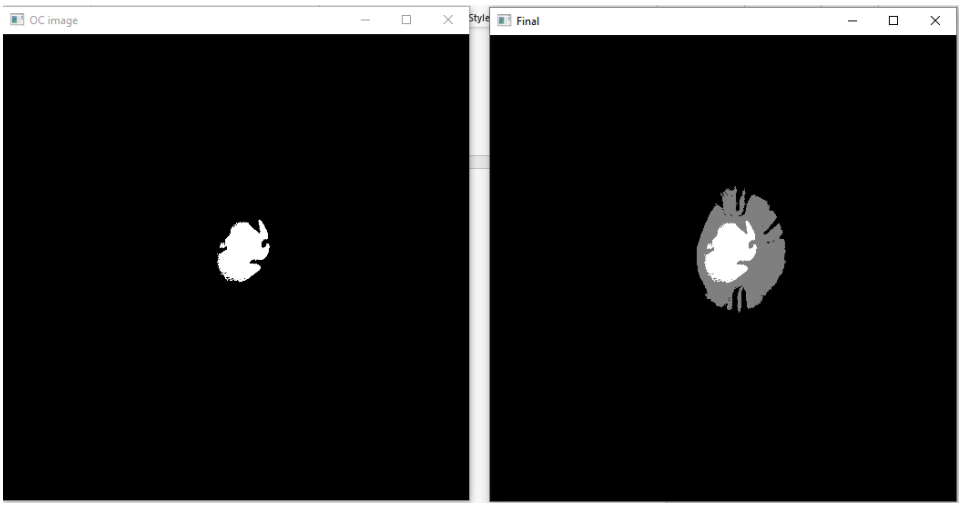


drishtiGS_097

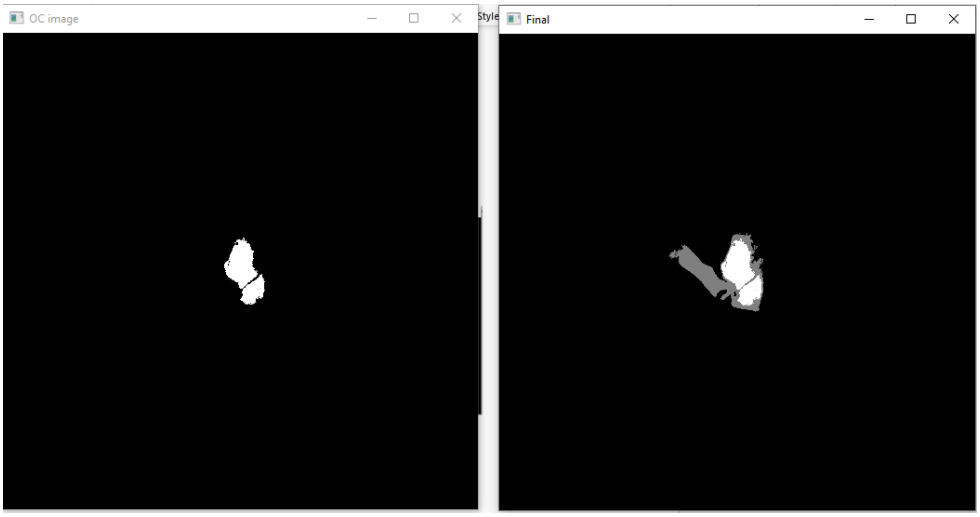


drishtiGS_099





drishtiGS_100



Dice Coefficient Output:

```
"G:\Computer Engineering\6th Semester\DIP\Assignment\Assignment1\
 .venv\Scripts\python.exe" "G:\Computer Engineering\6th
 Semester\DIP\Assignment\Assignment1\Phase3.py"
OD Threshold: 245.0
OC Threshold: 253.0
drishtiGS_007.png → BG Dice: 1.00,   OD Dice: 0.90,   OC Dice: 0.45
drishtiGS_009.png → BG Dice: 1.00,   OD Dice: 0.93,   OC Dice: 0.56
drishtiGS_013.png → BG Dice: 0.99,   OD Dice: 0.84,   OC Dice: 0.60
drishtiGS_072.png → BG Dice: 1.00,   OD Dice: 0.86,   OC Dice: 0.66
drishtiGS_078.png → BG Dice: 1.00,   OD Dice: 0.89,   OC Dice: 0.58
drishtiGS_085.png → BG Dice: 1.00,   OD Dice: 0.93,   OC Dice: 0.54
drishtiGS_091.png → BG Dice: 1.00,   OD Dice: 0.88,   OC Dice: 0.37
drishtiGS_093.png → BG Dice: 0.99,   OD Dice: 0.77,   OC Dice: 0.25
drishtiGS_095.png → BG Dice: 0.99,   OD Dice: 0.74,   OC Dice: 0.61
drishtiGS_096.png → BG Dice: 0.99,   OD Dice: 0.83,   OC Dice: 0.52
drishtiGS_097.png → BG Dice: 1.00,   OD Dice: 0.91,   OC Dice: 0.56
drishtiGS_099.png → BG Dice: 1.00,   OD Dice: 0.93,   OC Dice: 0.70
drishtiGS_100.png → BG Dice: 0.99,   OD Dice: 0.56,   OC Dice: 0.08

Average Dice of Test Images:
BG: 99.50%,   OD: 84.32%,   OC: 49.72%

Process finished with exit code 0
```